

LLD Learning Series - Day 5

Topic: Applying SOLID in Practice - Document Editor Case Study

The Problem Statement

Design a Document Editor that lets users add text and images to a document, render it, and save it. The design must be scalable so that new element types like tables, videos, new lines, tabs, and font customisation can be added in the future without modifying existing code.

Design Approach

Top-down: start from the highest-level object and break it down.

Bottom-up: start from smallest components and build upward. This is the preferred approach for this problem.

The Bad Design

The initial approach puts everything inside a single `DocumentEditor` class.

What it does

Stores both text and image elements together in one list of strings.

Differentiates images from text at render time by checking if the string ends with `.jpg` or `.png`.

`renderDocument()` iterates the list and builds one big output string.

`saveToFile()` writes that string directly to a file.

Code Snapshot: Bad Design

```
class DocumentEditor {
    vector<string> documentElements;
    void addText(string text);
    void addImage(string path);
    string renderDocument(); // checks .jpg/.png to detect images
    void saveToFile();
};
```

SOLID Violations in the Bad Design

Principle	Violation
SRP	One class handles adding, rendering, and saving. Three separate reasons to change.
OCP	Adding a new element type (e.g. video) requires modifying renderDocument() with another if-else.
DIP	No abstraction. DocumentEditor is tightly coupled to file-based saving and string-based rendering.
ISP	No interface separation. All functionality is crammed into one class.

The Improved Design

The refactored design separates concerns across focused classes and uses abstraction and polymorphism to make the system extensible.

Core Components

Component	Responsibility
DocumentElement (abstract)	Abstract base class with a pure virtual render() method. All element types inherit from this.
TextElement	Implements render() to return the text string.
ImageElement	Implements render() to return a formatted image path string.
NewLineElement	Implements render() to return a newline character.
TabSpaceElement	Implements render() to return a tab character.
Document	Holds a list of DocumentElement pointers. Manages add/remove. Its render() calls each element's render() via polymorphism.
Persistence (abstract)	Abstract class with a pure virtual save(string) method.
FileStorage	Implements save() by writing to a file.
DBStorage	Implements save() by writing to a database.
DocumentEditor	Client-facing class. Holds a Document and Persistence. Delegates all work to them.

Code Snapshot: Good Design

```
// Abstract element
class DocumentElement { virtual string render() = 0; };

// Concrete elements
class TextElement : public DocumentElement { string render() { return text; } };
class ImageElement : public DocumentElement { string render() { return "[Image:
" + path + "]; } };

// Document manages elements
class Document {
    vector<DocumentElement*> elements;
    void addElement(DocumentElement* e);
    string render(); // calls e->render() for each element
};

// Persistence abstraction
class Persistence { virtual void save(string data) = 0; };
class FileStorage : public Persistence { void save(string data) override { ... }
};

// Editor delegates everything
class DocumentEditor {
    Document* document;
    Persistence* storage;
    void addText(string t) { document->addElement(new TextElement(t)); }
    void addImage(string p) { document->addElement(new ImageElement(p)); }
    string renderDocument() { return document->render(); }
    void saveDocument() { storage->save(renderDocument()); }
};
```

SOLID Compliance in the Good Design

Principle	How it is Satisfied
SRP	Each class has one job: Document manages elements, each element renders itself, Persistence saves, DocumentEditor coordinates.
OCP	Adding a new element type (e.g. VideoElement) only requires a new subclass of DocumentElement. Nothing else changes.
LSP	Any DocumentElement subclass can be placed in the Document list and render() works correctly for all.
ISP	Persistence only exposes save(). DocumentElement only exposes render(). Interfaces are minimal and focused.
DIP	DocumentEditor depends on the abstract Document and Persistence, not on concrete FileStorage or TextElement.

Optional Further Improvement: DocumentRenderer

To push SRP even further, the render() logic can be extracted from Document into a separate DocumentRenderer class. Document then becomes purely a CRUD manager for elements, and DocumentRenderer takes a Document, retrieves its elements via a getter, and calls render() on each.

The Trade-off: Law of Demeter

The Law of Demeter (Principle of Least Knowledge) says an object should only talk to its immediate friends, not friends of friends.

When DocumentRenderer calls document.getElements() and then calls render() on each element, it is talking to a friend's friend (the elements inside Document). This is a violation. The practical resolution is often to keep render() inside Document itself, accepting the mild SRP trade-off in favour of avoiding indirect coupling.

Design Evolution Summary

Stage	What Changed	Problem Solved
Single DocumentEditor class	Everything in one class	Nothing yet. All SOLID violations present.
Add DocumentElement abstraction	Abstract base + TextElement, ImageElement subclasses	Fixes OCP and LSP. Rendering is polymorphic.
Add Document class	Document manages the element list and delegates render()	Fixes SRP for element management.
Add Persistence abstraction	FileStorage and DBStorage implement Persistence	Fixes DIP and ISP for saving.
Optional: DocumentRenderer	Rendering extracted from Document	Stricter SRP, but risks Law of Demeter violation.

My Takeaways from Today

1. The bad design was not obviously wrong. It worked. But it was fragile: adding a video element meant touching renderDocument() with another if-else check. That is OCP violation hiding in plain sight.
2. Polymorphism is the real fix for OCP here. Each element knows how to render itself. The Document just calls render() on each, not caring what type it is.
3. Seeing Document, Persistence, and DocumentEditor as three separate concerns made SRP click at a practical level. Each class has one job and one job only.
4. DIP in action: DocumentEditor receives a Persistence instance from outside. It never creates FileStorage itself. That is dependency injection, and it makes swapping storage trivial.

5. The Law of Demeter is a new principle. An object should only call methods on its own fields, not on objects returned by those fields.

6. There is no perfect design. The DocumentRenderer idea is cleaner in theory but introduces indirect coupling. The session reinforced that design principles are tools for thinking, not rules to blindly follow.